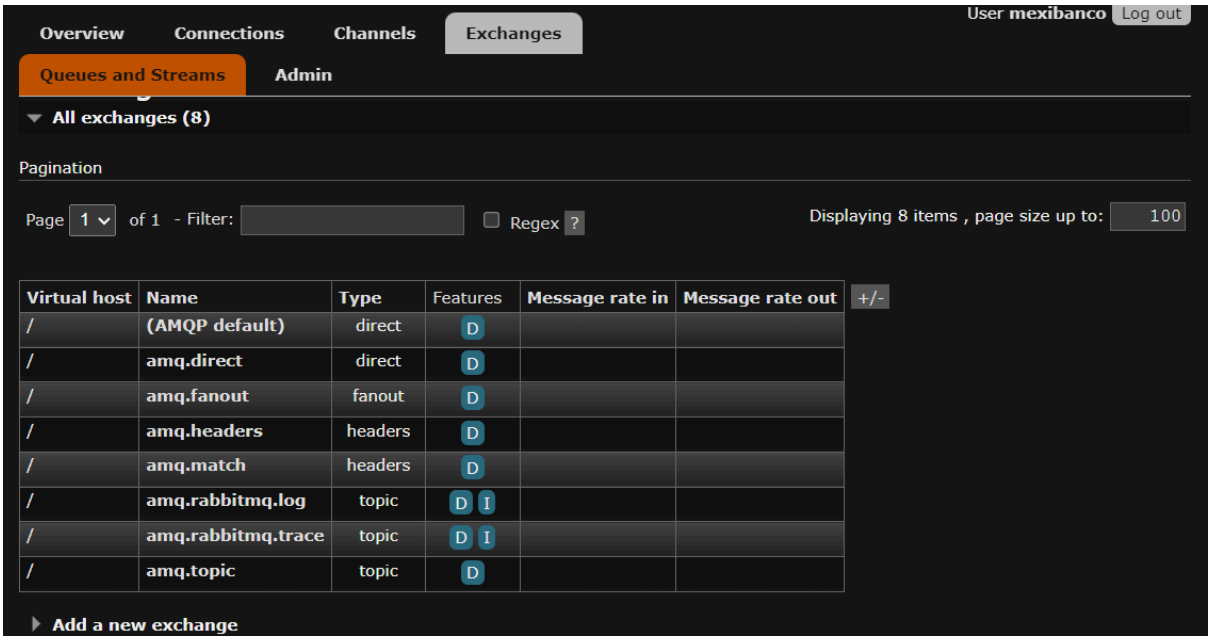
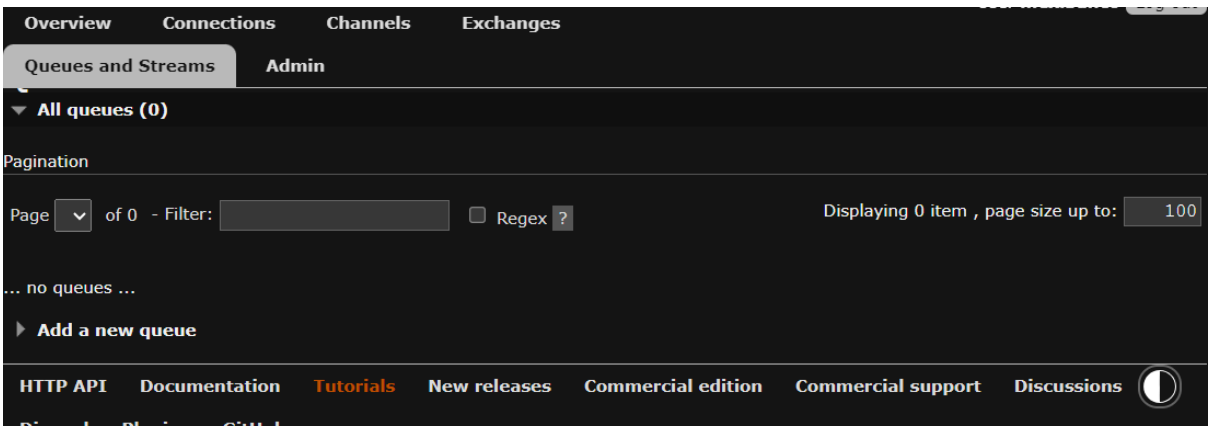
Bitácora Labs09: Que el aviso no tumbe la transferencia

1. El problema

1.1 RabbitMQ - Estado Inicial

Tras levantar la infraestructura con docker compose up, verifiqué que los 11 contenedores estuvieran en estado *healthy*. Al ingresar al panel de control de RabbitMQ (localhost:15672), revisé las pestañas de Exchanges y Queues. Como se observa en la captura, el broker está completamente limpio: aún no existen los exchanges ni las colas de nuestro dominio, lo que servirá como punto de partida para que nuestro código los declare automáticamente.





1.2 Preparación y Prueba de Error

Para iniciar las pruebas, se registraron las cuentas de Ana y Beto a través del API Gateway. Dependiendo de la persistencia de los volúmenes, la respuesta esperada era un 201 (creación) o un 409 (si ya existían).

```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s09 (05-mensajeria-base)
$ curl -i -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' \
  -d '{"clabe":"0021800000000000001","titular":"Ana","saldoInicial":1000}'
HTTP/1.1 201 Created
transfer-encoding: chunked
X-Instancia: 5e1c91bf18a6
Content-Type: application/json
Date: Thu, 22 Oct 2026 04:15:13 GMT

{"id":6,"clabe":"002180000000000001","titular":"Ana","saldo":1000}
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s09 (05-mensajeria-base)
$ curl -i -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' \
  -d '{"clabe":"0021800000000000002","titular":"Beto","saldoInicial":500}'
HTTP/1.1 201 Created
transfer-encoding: chunked
X-Instancia: 2369486b02a7
Content-Type: application/json
Date: Thu, 22 Oct 2026 04:15:26 GMT
```

Posteriormente, se procedió a simular una caída del servicio de notificaciones justo antes de emitir una transferencia de $50 de Ana a Beto. Como se observa en las capturas adjuntas, el cliente recibió un error 500, pero la consulta de saldos demuestra que el dinero sí fue descontado de origen y sumado en destino. La transferencia quedó permanentemente estancada en estado ABONADA.

Reflexión: ¿Por qué el cliente recibe un error si el dinero ya se movió? El cliente recibe un error porque la saga se quedó esperando la respuesta síncrona del servicio de notificaciones que estaba apagado, lo que provocó que el proceso tronara antes de poder marcar la transferencia como completada.

```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s09 (05-mensajeria-base)
$ docker compose stop notificacion
[+] stop 1/1
 ✔ Container mexi-banco-v06-notificacion-1 Stopped
                                    3.5s

emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s09 (05-mensajeria-base)
$ curl -i -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"cla
veOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":50}'
HTTP/1.1 500 Internal Server Error
transfer-encoding: chunked
Content-Type: application/json
Date: Thu, 22 Oct 2026 05:13:56 GMT

{"timestamp":"2026-10-22T05:13:56.293+00:00","status":500,"error":"Internal Server Error","path":"/trans
ferencias"}
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s09 (05-mensajeria-base)
$ curl -s 'http://localhost:8080/api/transferencias?estado=ABONADA'
[{"id":6,"claveOrigen":"002180009161154301","claveDestino":"002180009161154302","monto":50.00,"fecha":"2
026-10-21T00:46:08.117094Z","estado":"ABONADA","motivo":null,"saldoOrigenTrasCargo":850.00,"intentosDeCo
mpensacion":0,"actualizada":"2026-10-21T00:46:08.509387Z"},{"id":7,"claveOrigen":"002180000000000001","c
laveDestino":"002180000000000002","monto":50.00,"fecha":"2026-10-22T05:13:54.704120Z","estado":"ABONADA"
,"motivo":null,"saldoOrigenTrasCargo":950.00,"intentosDeCompensacion":0,"actualizada":"2026-10-22T05:13:
55.666752Z"}]
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s09 (05-mensajeria-base)
$ curl -s http://localhost:8080/api/cuentas/002180000000000001
curl -s http://localhost:8080/api/cuentas/002180000000000002
{"id":6,"clabe":"002180000000000001","titular":"Ana","saldo":950.00}{"id":7,"clabe":"002180000000000002"
,"titular":"Beto","saldo":550.00}
```

2. Publicar el evento

Para solucionar el problema de la transferencia estancada, comenzamos a implementar la publicación del evento en RabbitMQ. El primer paso en el código fue modificar la entidad Transferencia para agregarle un campo eventoId de tipo UUID. Esto es crucial porque nos sirve como "llave de idempotencia": garantiza que el identificador nazca junto con la

transferencia en la base de datos, evitando que el evento sea procesado como nuevo en caso de que ocurran reintentos de publicación por fallas en la red.

El Contrato (Evento) y Configuración del Broker

Creamos el registro **EventoTransferencia** para definir el contrato de comunicación (el mensaje que viajará por la red). Este evento representa un hecho en el pasado ("la transferencia ya se realizó" o "se revirtió") y se encarga de empaquetar los datos esenciales en formato JSON. Además, creamos la clase ConfiguracionMensajeria donde declaramos nuestro exchange de tipo *Topic* llamado transferencias y configuramos un Jackson2JsonMessageConverter. Esto es clave para que Spring Boot traduzca automáticamente nuestro objeto de Java a JSON usando el formato correcto de fechas antes de mandarlo a RabbitMQ.

Publisher Confirms y el Publicador de Eventos

Para evitar que los mensajes se pierdan silenciosamente si el broker falla por un milisegundo, modificamos el archivo application.yml activando publisher-confirm-type: correlated. Esto obliga a RabbitMQ a enviarnos un acuse de recibo (ACK o NACK). En el código clave, implementamos PublicadorDeEventos. Esta clase usa RabbitTemplate para publicar el evento en el exchange. Lo más importante aquí es que vinculamos el envío con un CorrelationData usando nuestro eventoId y obligamos al hilo a esperar la confirmación del broker (con un timeout). Si el broker no confirma, lanzamos nuestra propia EventoNoPublicadoException.

Desacoplamiento de la Saga y Prueba Final

Finalmente, editamos el orquestador (TransferenciaService). Eliminamos por completo el cliente HTTP síncrono de notificaciones y lo reemplazamos por nuestro PublicadorDeEventos. Ahora, al terminar los asientos contables, la saga simplemente publica el evento y se marca como COMPLETADA inmediatamente, sin importarle si el servicio de notificaciones está vivo o muerto.

Resultado de la prueba:

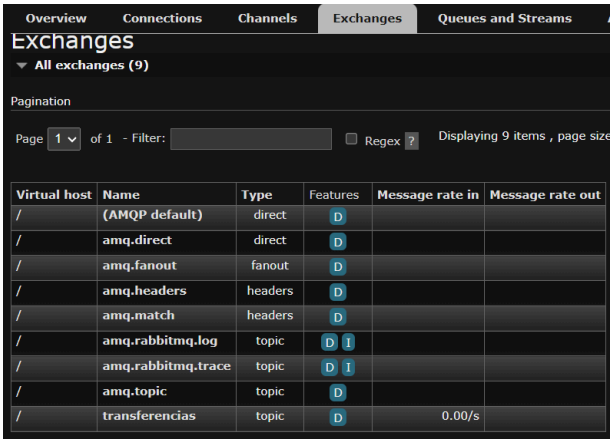
Volvimos a apagar el servicio de notificaciones (docker compose stop notificacion) y lanzamos una transferencia. Esta vez, la transferencia sí devolvió un HTTP 201 Created y quedó en estado COMPLETADA en la base de datos, adjuntando su propio eventoId. El sistema ya no se rompe; el mensaje quedó en RabbitMQ esperando a que notificaciones se recupere para ser procesado

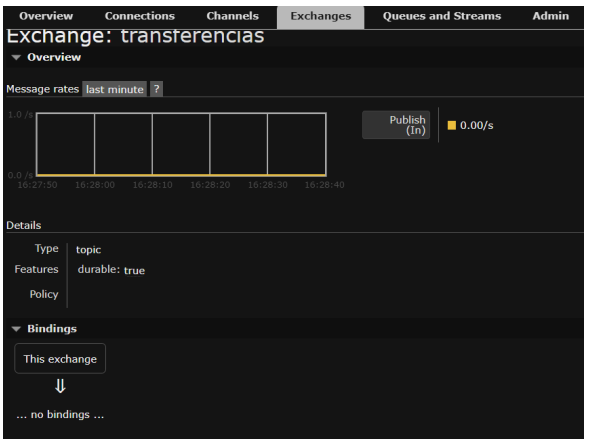
```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s09 (05-mensajeria-base)
$ docker compose stop notificacion
[+] stop 1/1
 ✔ Container mexi-banco-v06-notificac... Stopped 0.0s

emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s09 (05-mensajeria-base)
$ curl -i -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"0021800000000000001","claveDestino":"0021800000000000002","monto":50}'
HTTP/1.1 201 Created
transfer-encoding: chunked
Content-Type: application/json
Date: Thu, 22 Oct 2026 22:23:39 GMT

{"id":8,"claveOrigen":"0021800000000000001","claveDestino":"0021800000000000002","monto":50,"fecha":"2026-10-
22T22:23:37.8618264147","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":900.00,"intentosDeCompen
sacion":0,"actualizada":"2026-10-22T22:23:39.732880052Z","eventoId":"d2ea8b1c-0237-483c-8169-15f38bca7328"}
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s09 (05-mensajeria-base)
$ docker compose logs transferencia | grep evento
transferencia-1  | 2026-10-22T22:23:39.786Z ERROR 1 --- [transferencia] [nio-8080-exec-7] m.m.transferencia
.TransferenciaService    : Transferencia 8: quedo terminada pero el evento TransferenciaRealizada d2ea8b1c-0
237-483c-8169-15f38bca7328 NO se publico: Ninguna cola recibio el evento d2ea8b1c-0237-483c-8169-15f38bca73
28 con la routing key transferencia.realizada (NO_ROUTE)
```

Pestaña Exchanges donde aparece **Transferencia,** tipo topic, marca D & Bindings con ninguna cola entrelazada:





Reflexión: Publisher Confirms

- **¿Qué garantiza la confirmación?**

Garantiza únicamente que el broker (RabbitMQ) recibió el evento, lo persistió en disco y asumió la responsabilidad de cuidarlo.

- **¿Qué NO garantiza?**

No garantiza que el consumidor (el servicio de notificaciones) lo haya procesado o siquiera leído.

Análisis de los dos casos:

1. Si notificación está apagado: El broker de todas formas nos regresa un ack exitoso. El mensaje se queda guardado a salvo en la cola y la saga de la transferencia termina sin errores.

2. Si el proceso muere entre guardar(t) y avisar(...): La transferencia se queda guardada en la base de datos como COMPLETADA, pero como el proceso muere antes de lanzar el evento, el mensaje nunca llega a RabbitMQ y se pierde para siempre (el cliente nunca recibirá su aviso).

3. Consumir el evento (Servicio Notificación)

3.1 La cola y su binding

Para que los eventos no se pierdan cuando el *exchange* los reciba, configuré la infraestructura del lado del consumidor en ConfiguracionMensajeria.java. Declaré la cola notificacion.transferencias como *durable* mediante QueueBuilder, garantizando que sobreviva a reinicios del broker. Posteriormente, creé el *binding* para enlazar esta cola al exchange transferencias (de tipo *topic*) utilizando el patrón de ruteo transferencia.

3.2 El evento (Lado de notificación)

Para mantener el bajo acoplamiento entre microservicios, el servicio de notificación no importa las clases del servicio de transferencia. En su lugar, creé una copia local del contrato definiendo el record EventoTransferencia. Mientras el JSON entrante respete las propiedades acordadas (eventoId, tipo, transferenciaId, clabes y montos), la deserialización funcionará correctamente de manera independiente.

3.3 Guardar los avisos

Modifiqué NotificacionService.java para procesar el evento recibido. Dependiendo del tipo de evento, la lógica de negocio se adapta:

- Si es TransferenciaRealizada, se generan dos avisos (para la CLABE origen y destino).
- Si es TransferenciaRevertida (derivado de una saga compensada), solo se avisa a la CLABE origen sobre la devolución. Este bloque de código se anotó con @Transactional para garantizar atomicidad: los avisos del evento se guardan todos juntos o no se guarda ninguno, evitando estados inconsistentes en la base de datos de notificaciones.

3.4 El consumidor y el ACK manual

Para tener control total sobre cuándo RabbitMQ puede borrar un mensaje de la cola, cambié la configuración en application.yml a acknowledge-mode: manual, desactivando la confirmación automática de Spring. En la clase ConsumidorDeTransferencias implementé la lógica con @RabbitListener. El flujo es estricto: primero se lee el JSON a mano y se deserializa, luego se llama a notificaciones.avisarDeTransferencia(...) (donde ocurre el *commit* a la BD), y solo después de esto se ejecuta canal.basicAck(etiqueta, false). Adicionalmente, el log registra si el mensaje viene con la bandera redelivered=true.

3.5 Pruebas de desacople en tiempo

Comprobé físicamente el beneficio de la mensajería asíncrona:

1. Apagué el contenedor de notificación.
2. Lancé 3 transferencias exitosas (HTTP 201). El servicio principal no se vio afectado por la caída del notificador.
3. Verifiqué mediante rabbitmqctl y el panel de RabbitMQ que la cola notificacion.transferencias retuvo los 3 mensajes en estado *Ready*.
4. Al volver a encender notificación, el consumidor leyó y procesó los 3 mensajes acumulados al instante, registrando los avisos correctamente en las bandejas de Ana y Beto. Finalmente, probé el escenario de fallo de la S08 forzando un error en el abono; la transferencia regresó a estado COMPENSADA y comprobé que a Ana le llegó correctamente su notificación de DEVOLUCION_TRANSFERENCIA.

```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s09 (05-mensajeria-base)
$ for m in 11 12 13; do                          # 2. tres transferencias, de 11, 12 y 13
  curl -s -o /dev/null -w '%{http_code}\n' -X POST http://localhost:8080/api/transferencias \
    -H 'Content-Type: application/json' \
    -d "{\"claveOrigen\":\"002180000000000001\",\"claveDestino\":\"002180000000000002\",\"monto\":$m}"
done
201
201
201
```

```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s09 (05-mensajeria-base)
$ docker compose exec rabbitmq rabbitmqctl list_queues name messages_ready messages_unacknowledged
Timeout: 60.0 seconds ...
Listing queues for vhost / ...
name    messages_ready  messages_unacknowledged
notificacion.transferencias     3       0

emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s09 (05-mensajeria-base)
$ docker compose start notificacion
[+] start 4/4
 ✔ Container mexi-banco-v06-rabbitmq-1     Healthy
                                           0.5s
 ✔ Container mexi-banco-v06-db-1           Healthy
                                           0.5s
 ✔ Container mexi-banco-v06-discovery-1    Healthy
                                           0.5s
 ✔ Container mexi-banco-v06-notificacion-1 Started
                                           0.1s

emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s09 (05-mensajeria-base)
$ curl -s 'http://localhost:8080/api/notificaciones?clabe=002180000000000002'
[{"id":9,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 13, saldo resultante
636.00","fecha":"2026-10-23T00:23:39.371340Z"},{"id":7,"clabeCuenta":"002180000000000002","mensaje":"TR
ANSFERENCIA_RECIBIDA por 12, saldo resultante 623.00","fecha":"2026-10-23T00:23:39.365282Z"},{"id":5,"c
labeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 11, saldo resultante 611.00","fe
cha":"2026-10-23T00:23:39.288143Z"}]
```

Reflexión: ¿Por qué el ack va después de guardar y no antes?

El orden en que enviamos el *Acknowledge* (ack) define cómo responde el sistema ante un fallo (por ejemplo, si el servicio muere a la mitad de la ejecución).

- Si el ack fuera ANTES de guardar: RabbitMQ borraría el mensaje de la cola inmediatamente. Si el contenedor de notificación muere justo antes de poder hacer el *commit* en la base de datos, el evento se perdería para siempre en la red y el cliente jamás recibiría su aviso.
- Al poner el ack DESPUÉS de guardar (nuestra implementación): Si el servicio muere justo después de guardar en la base de datos pero antes de enviar el *ack* al broker,

RabbitMQ asumirá que el mensaje no se procesó. Cuando el servicio de notificación reviva, RabbitMQ le volverá a entregar el mensaje (marcado con redelivered=true). Esto significa que el aviso se repite en la base de datos, pero garantizamos que no se pierde.

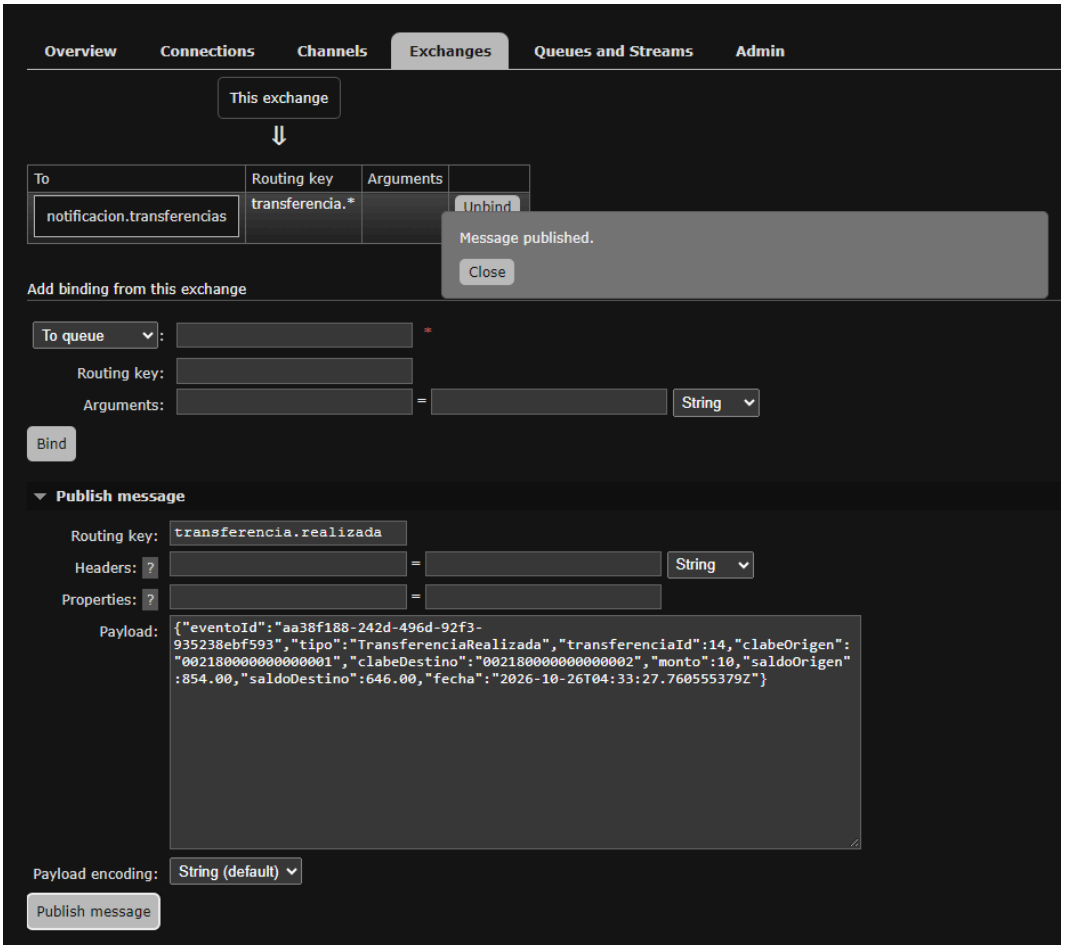
En un sistema bancario distribuido, es preferible lidiar con un aviso repetido que perder un mensaje de confirmación de transferencia silenciosamente.

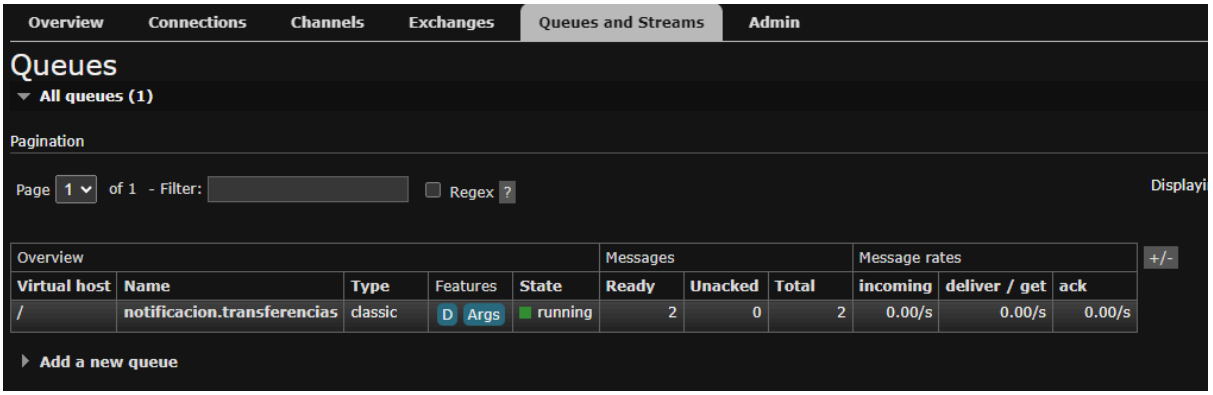
4: Tarea · El duplicado y la Idempotencia

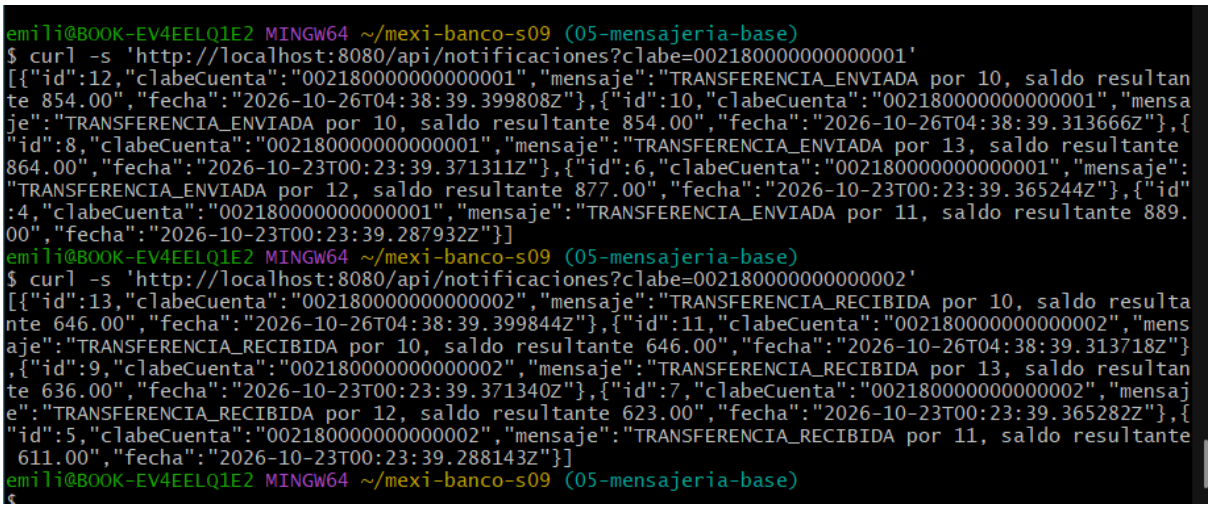
4.1 Provocar el duplicado (*At-least-once delivery*)

Como analizamos en la sección anterior, colocar la confirmación manual (basicAck) *después* de guardar en la base de datos nos garantiza que no se pierdan avisos en caso de caídas. Sin embargo, esto introduce un nuevo desafío arquitectónico: si el servicio de notificación procesa el evento y se apaga milisegundos antes de enviar el *ack*, RabbitMQ asume que el mensaje no se procesó y lo reencola.

Al provocar este escenario intencionalmente, el resultado fue que en la base de datos de Ana y Beto aparecieron notificaciones idénticas repetidas. Esto demuestra físicamente el comportamiento de entrega *at-least-once* (al menos una vez).





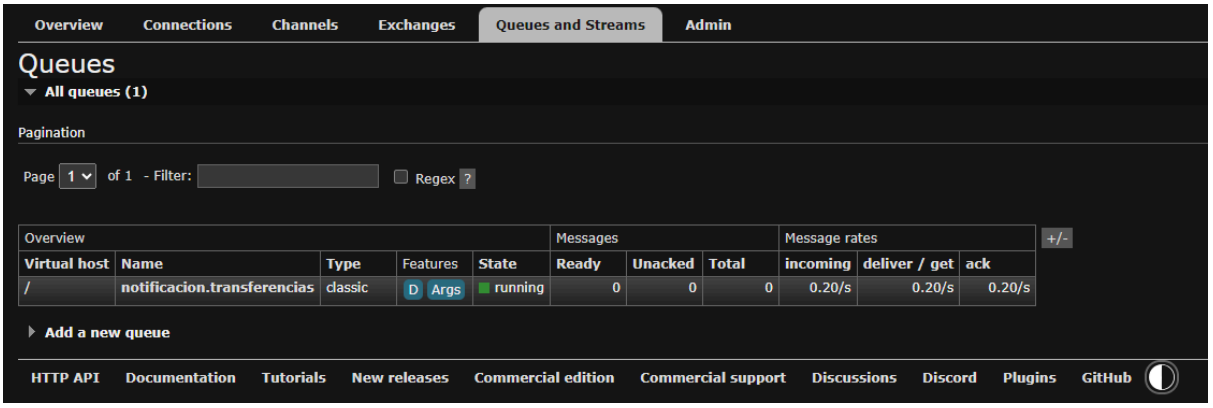


En la captura de la consola se observa la respuesta del *endpoint* de notificaciones para la cuenta origen (Ana). Se aprecian claramente dos registros distintos en la base de datos (con IDs diferentes) pero con el contenido exactamente idéntico: `"TRANSFERENCIA_ENVIADA por 10"`, generados en la misma fracción de segundo.

Esto materializa el problema del at-least-once delivery (entrega de al menos una vez). Al reinyectar el mismo mensaje (o simular la caída del consumidor antes de emitir el Acknowledge), RabbitMQ cumplió su trabajo de volver a entregarlo. Sin embargo, como el servicio de notificación actual no es **idempotente**, procesó el evento a ciegas por segunda vez, duplicando el aviso en la bandeja del cliente.

4.2 Solución: Consumidor Idempotente

Para mitigar este problema, es necesario que el consumidor sea idempotente (es decir, que procesar el mismo mensaje una o cien veces tenga el mismo resultado final). La clave para resolver esto es el eventoId (UUID) que viaja en el JSON. Como el servicio de transferencia genera este ID desde que nace la fila, el ID es exactamente el mismo aunque RabbitMQ reentregue el mensaje. Utilizando este identificador único en el servicio de notificaciones, se puede verificar en la base de datos si el evento ya fue procesado antes de insertar un aviso nuevo, descartando así los duplicados silenciosamente y enviando el *ack* correspondiente.




```
notificacion-1  | 2026-10-26T05:19:53.732Z  INFO 1 --- [notificacion] [rap-executor-%d] c.n.d.s.r.aw
s.ConfigClusterResolver      : Resolving eureka endpoints via configuration
notificacion-1  | 2026-10-26T05:21:23.777Z  WARN 1 --- [notificacion] [pool-1-thread-5] o.s.a.r.list
ener.BlockingQueueConsumer   : Cancel received for amq.ctag-wI-6W4hSS1Gt3KtiFigfJQ (notificacion.tra
nsferencias); Consumer@2d0f850: tags=[[amq.ctag-wI-6W4hSS1Gt3KtiFigfJQ]], channel=Cached Rabbit Chan
nel: AMQChannel(amqp://mexibanco@172.20.0.4:5672/,1), conn: Proxy@286e2b71 Shared Rabbit Connection:
 SimpleConnection@4b27a3fc [delegate=amqp://mexibanco@172.20.0.4:5672/, localPort=33700], acknowledg
eMode=MANUAL local queue size=0
notificacion-1  | 2026-10-26T05:21:23.862Z  WARN 1 --- [notificacion] [ntContainer#0-1] o.s.a.r.l.Si
mpleMessageListenerContainer : Consumer raised exception, processing can restart if the connection f
actory supports it. Exception summary: org.springframework.amqp.rabbit.support.ConsumerCancelledExce
ption
notificacion-1  | 2026-10-26T05:21:23.867Z  INFO 1 --- [notificacion] [ntContainer#0-1] o.s.a.r.l.Si
mpleMessageListenerContainer : Restarting Consumer@2d0f850: tags=[[]], channel=Cached Rabbit Channel
: AMQChannel(amqp://mexibanco@172.20.0.4:5672/,1), conn: Proxy@286e2b71 Shared Rabbit Connection: Si
mpleConnection@4b27a3fc [delegate=amqp://mexibanco@172.20.0.4:5672/, localPort=33700], acknowledgeMo
de=MANUAL local queue size=0
notificacion-1  | 2026-10-26T05:23:45.583Z  INFO 1 --- [notificacion] [ntContainer#0-2] m.m.n.m.Cons
umidorDeTransferencias       : [consumidor] TransferenciaRealizada de la transferencia 14 (evento aa
38f188-242d-496d-92f3-935238ebf593, redelivered=false): 0 aviso(s) nuevo(s)

emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s09 (05-mensajeria-base)
$ curl -s 'http://localhost:8080/api/notificaciones?clabe=002180000000000002'
[{"id":15,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 10, saldo resulta
nte 646.00","fecha":"2026-10-26T05:12:02.622738Z"},{"id":13,"clabeCuenta":"002180000000000002","mens
aje":"TRANSFERENCIA_RECIBIDA por 10, saldo resultante 646.00","fecha":"2026-10-26T04:38:39.399844Z"}
,{"id":11,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 10, saldo resulta
nte 646.00","fecha":"2026-10-26T04:38:39.313718Z"},{"id":9,"clabeCuenta":"002180000000000002","mensa
je":"TRANSFERENCIA_RECIBIDA por 13, saldo resultante 636.00","fecha":"2026-10-23T00:23:39.371340Z"},
{"id":7,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 12, saldo resultant
e 623.00","fecha":"2026-10-23T00:23:39.365282Z"},{"id":5,"clabeCuenta":"002180000000000002","mensaje
":"TRANSFERENCIA_RECIBIDA por 11, saldo resultante 611.00","fecha":"2026-10-23T00:23:39.288143Z"}]
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s09 (05-mensajeria-base)
$ curl -s 'http://localhost:8080/api/notificaciones?clabe=002180000000000001'
[{"id":14,"clabeCuenta":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 10, saldo resultan
te 854.00","fecha":"2026-10-26T05:12:02.622342Z"},{"id":12,"clabeCuenta":"002180000000000001","mensa
je":"TRANSFERENCIA_ENVIADA por 10, saldo resultante 854.00","fecha":"2026-10-26T04:38:39.399808Z"},{
"id":10,"clabeCuenta":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 10, saldo resultante
 854.00","fecha":"2026-10-26T04:38:39.313666Z"},{"id":8,"clabeCuenta":"002180000000000001","mensaje"
:"TRANSFERENCIA_ENVIADA por 13, saldo resultante 864.00","fecha":"2026-10-23T00:23:39.371311Z"},{"id
":6,"clabeCuenta":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 12, saldo resultante 877
.00","fecha":"2026-10-23T00:23:39.365244Z"},{"id":4,"clabeCuenta":"002180000000000001","mensaje":"TR
ANSFERENCIA_ENVIADA por 11, saldo resultante 889.00","fecha":"2026-10-23T00:23:39.287932Z"}]
```

5: Tarea · El veneno y la Dead Letter Queue (DLQ)

5.1 El problema de los mensajes envenenados

En los sistemas de mensajería asíncrona, un riesgo latente es la recepción de un mensaje con formato incorrecto, datos corruptos o esquemas desconocidos (*poison message*). Si el consumidor intenta procesar este tipo de carga útil y simplemente lanza una excepción sin confirmar el mensaje, el *broker* lo reencolará infinitamente, consumiendo recursos de cómputo y bloqueando el procesamiento de mensajes válidos en la cola.

5.2 Configuración de la topología con Dead Letter Exchange (DLX)

Para aislar estos fallos de forma segura, implementé el patrón *Dead Letter Queue* actualizando ConfiguracionMensajeria.java:

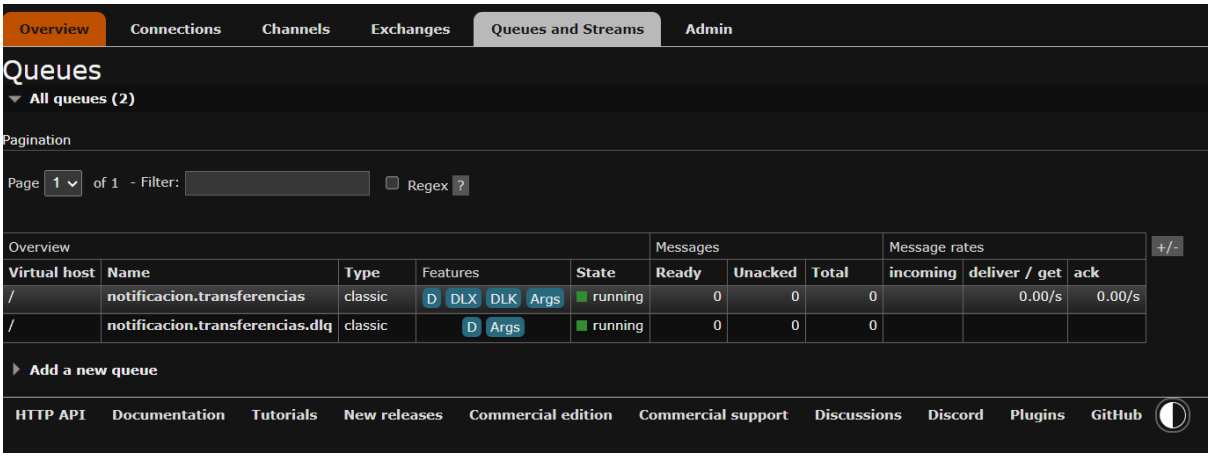
1. Añadí los argumentos de infraestructura a la cola principal (QueueBuilder.durable(...)) para redirigir los mensajes rechazados hacia un nuevo *exchange* de tipo *direct* llamado transferencias.dlx.
2. Declaré la cola de respaldo notificacion.transferencias.dlq y establecí su respectivo *binding*.

(Nota de infraestructura: Para aplicar esta modificación en el broker sin que arrojara un error de tipo PRECONDITION_FAILED por alterar una cola existente, fue necesario eliminar previamente la cola notificacion.transferencias desde el panel de administración de RabbitMQ y reiniciar el contenedor con docker compose up --build -d notificacion).

5.3 Rechazo definitivo y aislamiento del veneno

En la lógica del consumidor (ConsumidorDeTransferencias.java), envolví la lectura del mensaje en un bloque try-catch. Ante un error de deserialización o fallo irrecuperable al procesar el evento, se ejecuta un *Negative Acknowledge*:canal.basicNack(etiqueta, false, false)

Al configurar el último parámetro en false, se le indica a RabbitMQ que **no reencole** el mensaje. En su lugar, el *broker* lo transfiere automáticamente a la *Dead Letter Queue* (notificacion.transferencias.dlq), dejando la cola principal limpia y operativa para continuar procesando las transferencias sanas



Preguntas de Cierre

¿Qué sucede con el aviso si RabbitMQ está caído al momento en que la saga alcanza el estado `COMPENSADA` o `COMPLETADA` en tu implementación actual, y por qué el uso de *publisher confirms* no resuelve este escenario específico?
En nuestra arquitectura actual, si el broker no está disponible al momento de emitir el evento, la llamada síncrona o el intento de publicación del mensaje genera una excepción que sale del flujo principal. Aunque el patrón de *publisher confirms* garantiza que el mensaje llegó de manera segura al *exchange* de RabbitMQ, no protege contra la caída del *broker*

previa a la publicación ni asegura que el consumidor final (el servicio de notificaciones) lo procese y persista correctamente en su base de datos.

¿Cuál es la justificación técnica de colocar la confirmación manual (`basicAck`) posterior al almacenamiento en la base de datos y no antes, y qué nivel de garantía de entrega (*at-least-once* vs *at-most-once*) se obtiene con cada enfoque?
Colocar el `basicAck` después de persistir el aviso en la base de datos asegura que, si el microservicio de notificaciones colapsa milisegundos antes de confirmar, RabbitMQ no descarte el mensaje y lo reencole. Esto implementa una garantía de entrega de *al menos una vez* (*at-least-once*). Por el contrario, emitir el *ack* antes de guardar ofrecería una garantía de *a lo sumo una vez* (*at-most-once*), arriesgando la pérdida definitiva de los avisos ante cualquier falla repentina.

Análisis de compromisos arquitectónicos (Trade-offs):

- Elección: Desacoplar la notificación de la transferencia mediante mensajería asíncrona.
- Porque: Al eliminar la llamada HTTP síncrona del flujo crítico, evitamos que una caída o lentitud en el servicio de notificaciones bloquee, revierta o propague fallos hacia el servicio transaccional de transferencias.
- Sacrificio: Se pierde la consistencia fuerte inmediata y la retroalimentación síncrona para el cliente, aceptando consistencia eventual y el desafío operativo de gestionar duplicados, reintentos y mensajes envenenados (*Dead Letter Queues*).